

**Smt. Chandaben Mohanbhai Patel Institute of Computer
Applications, CHARUSAT**

**BCA Semester-III
CAUE302: Python Programming**

Practical 5: Mapping and Other Data Types (Dictionaries and Sets)

1. Introduction to Mapping and Collection Types

Python provides powerful built-in non-sequence data structures: Dictionaries and Sets. A Dictionary (dict) is Python's standard mapping type that stores elements as key-value pairs, where keys must be unique and immutable. A Set (set) is an unordered, mutable collection of unique, diverse elements. Mastering dictionary lookup operations and mathematical set theory operations (Union, Intersection, Difference, Symmetric Difference) is essential for real-world application logic and data processing.

2. Part A: Python Dictionary (dict)

A Dictionary is a collection of key-value pairs enclosed in curly braces {}. Key Characteristics:

- Key-Value Pair Mapping: Data is stored as key: value.
- Unique & Immutable Keys: Keys must be unique and hashable (e.g., strings, ints, tuples). Duplicate keys overwrite existing values.
- Dynamic & Mutable: You can add, update, or remove key-value pairs dynamically.

- Fast Lookup: Values are retrieved instantly using their keys rather than numerical index searching.

Topic 1: Dictionary Creation, Accessing Elements & Traversal

Access values using square brackets `dict[key]`, the safe `.get(key)` method, or iterate using `.keys()`, `.values()`, and `.items()`.

Syntax Structure:

Accessing Syntax

```
val = my_dict['key'] # Raises KeyError if missing
```

```
val = my_dict.get('key', default_val) # Returns None/default if missing
```

Complete Example 1: Dictionary Creation and Accessing Methods:

```
ele_dict = {'clgname': 'charusat', 'batch': 2024, 'branch': 'BCA'}
```

```
print('Access via []:', ele_dict['clgname'])
```

```
print('Access via get():', ele_dict.get('batch'))
```

Complete Example 2: Iterating Through Keys, Values, and Items:

```
student = {'roll': 101, 'name': 'Krupa', 'course': 'BCA'}
```

```
print('--- Iterating Keys ---')
```

```
for k in student.keys():
```

```
    print('Key:', k)
```

```
print('--- Iterating Values ---')
```

```
for v in student.values():
```

```
    print('Value:', v)
```

```
print('--- Iterating Key-Value Pairs ---')
```

```
for key, value in student.items():
```

```
    print(f'{key} -> {value}')
```

Topic 2: Modifying, Adding & Removing Dictionary Elements

Add or update elements using direct assignment `dict[key] = val` or `.update()`. Remove elements using `del`, `.pop()`, `.popitem()`, or `.clear()`.

Syntax Structure:

```
my_dict['new_key'] = value
my_dict.update({'k1': 'v1', 'k2': 'v2'})
val = my_dict.pop('key')
last_pair = my_dict.popitem()
```

Complete Example 1: Adding and Updating Dictionary Elements:

```
ele_dict = {'clgname': 'charusat', 'batch': 2024}
ele_dict['branch'] = 'BCA' # Direct Addition
ele_dict['batch'] = 2025 # Direct Modification
print('After Direct Updates:', ele_dict)
ele_dict.update({'city': 'Anand', 'state': 'Gujarat'})
print('After update():', ele_dict)
```

Complete Example 2: Deleting Dictionary Elements (del, pop, popitem, clear):

```
data = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
del data['a'] # Removes key 'a'
print('After del data["a"]:', data)
val = data.pop('b') # Removes 'b' and returns its value
print('Popped Value:', val, '| Dict:', data)
last_tuple = data.popitem() # Removes last key-value pair as tuple
print('Popped Last Item:', last_tuple, '| Dict:', data)
data.clear() # Removes all elements
print('After clear():', data)
```

3. Part B: Python Set (set)

A Set is an unordered collection of unique elements enclosed within curly braces {} or created using set(). Key Characteristics:

- Unordered & No Indexing: Items have no fixed position; indexing set[0] and slicing set[1:3] are NOT supported.
- No Duplicates: Duplicate elements are automatically ignored.
- Diversity of Elements: Can store integers, floats, strings, or tuples (must be immutable/hashable).
- Mutable: Elements can be added or removed dynamically.

Topic 3: Set Creation, Membership Testing & Modification

Create sets using {} or set(). Check existence using 'in' / 'not in'. Add items via .add() or .update(). Remove items via .remove() or .discard().

Syntax Structure:

Empty Set Creation

```
empty_set = set()
```

Adding & Removing

```
my_set.add(item)
```

```
my_set.update([i1, i2])
```

```
my_set.remove(item) # Raises KeyError if missing
```

```
my_set.discard(item) # No error if missing
```

Complete Example 1: Set Creation, Membership Testing & Duplicate Removal:

```
ele_set = {1, 2, 3, 4, 5, 2, 3} # Duplicates automatically removed
```

```
print('Set Contents:', ele_set)
```

```
print('Is 3 in set?:', 3 in ele_set)
```

```
print('Is 10 in set?:', 10 in ele_set)
```

Complete Example 2: Adding and Removing Elements (add, update, remove, discard):

```
s = {10, 20, 30}
s.add(40) # Add single element
s.update({50, 60}) # Add multiple elements from iterable
print('After add & update:', s)
s.remove(20) # Removes 20 (Raises error if missing)
s.discard(99) # Safe remove (No error if missing)
print('After remove & discard:', s)
```

Topic 4: Mathematical Set Operations (Union, Intersection, Difference)

Sets support core mathematical set theory operations using operators ($|$, $\&$, $-$) or built-in methods.

Syntax Structure:

```
union_set = a.union(b) # or a | b
inter_set = a.intersection(b) # or a & b
diff_set = a.difference(b) # or a - b
```

Complete Example 1: Mathematical Set Operations:

```
a_set = {1, 2, 4, 5}
b_set = {3, 4, 5, 6, 7}
print('Set A:', a_set)
print('Set B:', b_set)
print('Union (A | B):', a_set.union(b_set))
print('Intersection (A & B):', a_set.intersection(b_set))
print('Difference (A - B):', a_set.difference(b_set))
```